



Aplicativo mobile offline-first para registro de treino, composição corporal e nutrição.

DOCUMENTO

Descrição de projeto — desenvolvimento mobile

AUTORES

Paulo Rogério
Assis Damasceno

PLATAFORMA

Android e iOS (React Native / Expo)

DATA

Agosto de 2026

STACK DO APP

Expo SDK 57 · React Native 0.86 · Expo Router 57
expo-sqlite + Drizzle ORM · NativeWind

BACKEND

Supabase (Postgres + Auth + Storage)
OpenStreetMap / Overpass API

O Px GYM é usado no pior lugar possível para depender de rede: dentro da academia, entre uma série e outra, com uma mão só. Por isso o aplicativo foi construído com o banco local como fonte da verdade durante o treino — cada série marcada é gravada no SQLite do aparelho e só depois empurrada para o Supabase, quando houver conexão. Este documento descreve o escopo, a navegação, a estratégia offline-first, o backend, o uso de câmera e GPS e a arquitetura de código do projeto.

- 1 Visão geral e escopo
- 2 Interface e navegação · Expo Router
- 3 Estratégia offline-first · SQLite
- 4 Backend e sincronização · Supabase
- 5 Integração com hardware e sensores

01 Visão geral e escopo

Tema e objetivo

O Px GYM reúne em um único aplicativo três informações que hoje vivem separadas na rotina de quem treina: **o registro do treino** (qual carga foi usada em cada série), **a evolução da composição corporal** (peso, percentual de gordura, massa magra, medidas) e **as metas nutricionais** (calorias e macronutrientes). O objetivo é que o usuário consiga responder “estou evoluindo?” em poucos toques e registrar a série do dia sem fricção.

O problema central não é apenas de organização, é de **contexto de uso**. O momento em que o dado precisa ser registrado — logo depois de terminar a série — é exatamente o momento em que o usuário está em pé, suado, com o celular em uma mão e, com frequência, sem sinal: sala de peso livre no subsolo, Wi-Fi da academia sobrecarregado, plano de dados no fim do mês. Um app que exija rede para salvar uma série simplesmente não é usável nesse cenário. Daí a decisão que organiza todo o resto do projeto: **o registro do treino funciona 100% offline**, e a nuvem é um destino eventual, nunca um pré-requisito.

Um segundo problema que o app resolve é a **progressão de carga**. Planilhas e apps genéricos guardam o histórico, mas obrigam o usuário a garimpar “quanto eu levantei da última vez?”. No Px GYM, a carga anterior de cada exercício é lida do banco local e mostrada junto ao campo de registro, e a curva de progressão daquele exercício é desenhada na mesma tela.

Público-alvo

O usuário final é o **praticante individual de musculação** que acompanha o próprio progresso: treina com regularidade, faz avaliação física periódica (a cada algumas semanas) e segue uma meta de macros. Ele é o único operador do sistema — não existe, nesta fase, papel de treinador, de nutricionista ou de administrador, e nenhum usuário enxerga dados de outro. Essa decisão de escopo aparece diretamente na segurança do backend: todas as tabelas têm Row Level Security ligada com política de dono (`auth.uid() = user_id`), conforme a seção 4.

O mesmo usuário, com a mesma conta, também acessa uma aplicação web complementar (Next.js) usada nos momentos de planejamento — é por lá que ele envia o PDF da avaliação física, revisa os dados extraídos e gera o plano nutricional. O aplicativo mobile **consume** esse resultado; ele não é o lugar de fazer upload de PDF nem de revisar laudo.

Casos de uso principais

A coluna “offline” indica o comportamento sem conexão — a justificativa de cada caso está na seção 3.

CASO DE USO	O QUE O USUÁRIO FAZ	OFFLINE
Entrar ou criar conta	Autentica com e-mail e senha; a sessão fica guardada cifrada no aparelho.	só reabrir
Ver o dia	Abre o app e vê sequência de dias treinados, faixa da semana, treino do dia e resumo corporal.	parcial
Executar o treino do dia	Percorre a lista de exercícios e acompanha quantas séries já fechou.	sim
Registrar uma série	Ajusta a carga, marca a série concluída e dispara o cronômetro de descanso.	sim
Desmarcar uma série	Corrige um toque errado; o descanso é cancelado e o registro é removido.	sim
Ver progressão de carga	Consulta a curva de carga do exercício e a comparação com a sessão anterior.	sim
Abrir exercício pelo QR	Aponta a câmera para o QR code do aparelho e cai direto na tela daquele exercício.	sim
Registrar foto de progresso	Tira a foto pela câmera do app; ela é salva no aparelho na hora.	sim
Ver a galeria de fotos	Revisa as fotos por data e vê quais já subiram para a nuvem.	sim
Acompanhar composição corporal	Vê gráficos de peso, gordura e massa magra e a tabela de medidas por avaliação.	não
Consultar metas de macros	Vê calorias-alvo e proteína/carboidrato/gordura do plano vigente.	não
Encontrar academias por perto	Usa o GPS para listar e mapear academias num raio de 4 km.	não
Conferir a sincronização	Vê quantos registros ainda estão pendentes e força um envio manual.	sim

Fora de escopo nesta fase. Rede social ou compartilhamento, biblioteca aberta de exercícios, montagem de treino dentro do app (a rotina do dia ainda é fixa, definida em código), registro de refeições consumidas (o app mostra a meta, não o consumido) e qualquer papel além do próprio usuário.

02 Interface e navegação

A navegação é organizada em três camadas do Expo Router: um **Stack raiz** que decide entre área pública e área logada, um **Drawer** que guarda os destinos secundários e as telas de tela cheia, e um **Tabs** com os quatro destinos do dia a dia.

Mapeamento de telas

TELA	ROTA / ARQUIVO	PAPEL
Login	(auth)/login	E-mail e senha; entrada padrão de quem não tem sessão.
Cadastro	(auth)/cadastro	Criação de conta no Supabase Auth.
Início	(app)/(tabs)/index	Sequência de dias, faixa da semana, treino do dia, resumo corporal e macros.
Treino do dia	(app)/(tabs)/treino/index	Lista de exercícios com progresso de séries e atalho para o leitor de QR.
Exercício	(app)/(tabs)/treino/[exercicioId]	Série a série: carga, marcação, descanso e curva de progressão.
Nutrição	(app)/(tabs)/nutricao	Calorias-alvo e macros do plano vigente.
Perfil	(app)/(tabs)/perfil	Identidade da conta e atalhos para as telas do menu.
Progresso	(app)/progresso	Gráficos de peso, gordura e massa magra; tabela de medidas.
Avaliações	(app)/avaliacoes/index	Histórico de avaliações com status de processamento.
Fotos de progresso	(app)/fotos	Galeria local por data, com indicador de sincronização.
Câmera	(app)/camera	Captura da foto de progresso (tela cheia).
Leitor de QR	(app)/scanner	Leitura do QR do aparelho (tela cheia).
Academias por perto	(app)/academias	Mapa e lista de academias próximas pelo GPS.
Configurações	(app)/configuracoes	Conta, estado da sincronização, envio manual e versão.

Estrutura de roteamento

O Expo Router deriva as rotas da árvore de arquivos. Grupos entre parênteses — `(auth)`, `(app)`, `(tabs)` — organizam a hierarquia de layouts **sem** aparecer na URL, o que permite separar público de privado sem poluir os caminhos.

```
src/app/_layout.tsx      Stack raiz · AuthProvider · migrations do SQLite
|                          e as rotas protegidas
├─ (auth)/_layout.tsx    Stack público – só monta sem sessão
|   ├─ login.tsx
|   └─ cadastro.tsx
└─ (app)/_layout.tsx     Drawer – só monta com sessão
    ├─ (tabs)/_layout.tsx Tabs (barra flutuante própria)
    |   ├─ index.tsx      Início
    |   └─ treino/_layout.tsx Stack aninhada (fluxo sequencial)
    |       ├─ index.tsx  lista de exercícios
    |       └─ [exercicioId].tsx rota dinâmica
    └─ [exercicioId].tsx
        ├─ nutricao.tsx
        └─ perfil.tsx
    ├─ progresso.tsx
    ├─ avaliacoes/_layout.tsx Stack (abre espaço para o detalhe)
    │   └─ index.tsx
    ├─ fotos.tsx
    ├─ academias.tsx
    ├─ configuracoes.tsx
    ├─ camera.tsx        tela cheia · swipeEnabled: false
    └─ scanner.tsx       tela cheia · swipeEnabled: false
```

Figura 1. Árvore de rotas. Cada `_layout.tsx` instala um navegador; os arquivos irmãos viram telas dele.

Por que cada padrão foi escolhido

- **Tabs para os quatro destinos do dia a dia** (Início, Treino, Nutrição, Perfil): são as telas alcançadas dezenas de vezes por semana e precisam estar a um toque, com o polegar, sem abrir menu.
- **Stack aninhada dentro da aba Treino**: lista → exercício é um fluxo sequencial com volta natural. Aninhar a Stack dentro da aba preserva a barra de abas e mantém o histórico do fluxo isolado das outras abas.
- **Drawer para os destinos secundários** (Progresso, Avaliações, Fotos, Academias, Configurações): telas de consulta, visitadas com frequência baixa. Colocá-las em abas competiria com o fluxo de treino.
- **Telas de tela cheia no Drawer com gesto desativado**: câmera e leitor de QR ocupam a tela inteira e usam gestos próprios — o swipe lateral do Drawer é desligado nelas

(`swipeEnabled: false`) para não roubar o toque.

Rotas protegidas

A proteção é declarativa, no layout raiz: `Stack.Protected` monta o grupo `(app)` apenas quando existe sessão e o grupo `(auth)` apenas quando não existe. Não há redirecionamento manual espalhado pelas telas — quem sai da conta simplesmente vê o grupo protegido desmontar, e qualquer tela interna deixa de estar no ar.

```
// src/app/_layout.tsx — a sessão vem do AuthProvider (Supabase Auth)
<Stack screenOptions={{ headerShown: false }}>
  <Stack.Protected guard={!session}>
    <Stack.Screen name="(app)" />
  </Stack.Protected>
  <Stack.Protected guard={!session}>
    <Stack.Screen name="(auth)" />
  </Stack.Protected>
</Stack>
```

O mesmo layout raiz também segura a splash nativa até três coisas estarem prontas: fontes carregadas, sessão lida do armazenamento seguro e **migrations do SQLite aplicadas**. Nenhuma tela toca o banco antes disso.

Links profundos

O app registra o esquema `pxgym: //` e usa rotas tipadas (`typedRoutes`), o que faz o TypeScript validar cada caminho de navegação em tempo de compilação. O esquema é o que permite o QR code do aparelho apontar para `pxgym://treino/<exercicioId>` e abrir a tela do exercício direto — detalhado na seção 5.

03 Estratégia offline-first

Durante o treino, o SQLite do aparelho é a fonte da verdade. O app nunca espera resposta de rede para confirmar uma ação do usuário: grava local, redesenha a tela e trata o envio para a nuvem como tarefa de segundo plano.

Funcionamento offline

Sem conexão, o usuário abre o app já autenticado (a sessão fica cifrada no aparelho) e faz o treino inteiro:

- Percorre o treino do dia, marca e desmarca séries, ajusta a carga e usa o cronômetro de descanso.
- Vê a **carga anterior** de cada exercício e a curva de progressão — ambas calculadas a partir do próprio banco local, não de um endpoint.
- Vê a sequência de dias treinados e a faixa dos últimos sete dias, recalculadas a cada série marcada.
- Tira fotos de progresso e navega pela galeria: os arquivos moram no aparelho, então a galeria abre igual com ou sem rede.
- Confere em Configurações quantos registros e fotos ainda estão pendentes de envio, e pode tentar sincronizar manualmente.

Três telas **dependem de rede** por natureza, e isso é uma decisão explícita, não um esquecimento: **Avaliações** e **Nutrição** leem dados que nascem no aplicativo web (o laudo é enviado e processado lá), e **Academias por perto** consulta a base do OpenStreetMap em tempo real. Nesses casos o app não mostra tela vazia nem spinner infinito: cada uma tem estado de erro próprio, com ícone de “sem nuvem” e texto explicando que os dados voltam quando a conexão voltar.

Critério usado para decidir o que é offline. É offline aquilo que o usuário **produz** no aparelho durante o treino — porque perder esse dado significa perder o treino. É online aquilo que ele apenas **consulta** e que muda em outro lugar (laudo, plano nutricional, mapa de academias) — cachear isso traria complexidade de invalidação sem resolver o problema real.

Modelagem de dados local

O banco local é um `expo-sqlite` chamado `pxgym.db`, acessado pelo **Drizzle ORM** com esquema tipado e migrations versionadas (aplicadas no boot). Duas tabelas sustentam o modo offline:

set_logs — registros de série

COLUNA	TIPO	POR QUE PRECISA ESTAR NO APARELHO
id	text (PK)	UUID gerado no cliente — é o mesmo id usado no Postgres, o que torna o envio idempotente.
session_date	text	Dia da sessão no fuso do aparelho; base da sequência de dias e da faixa semanal.
exercise_id exercise_name	text	O nome é copiado junto para que o histórico continue legível offline, sem depender de catálogo remoto.
set_number target_reps	integer	Posição da série no exercício e repetições-alvo daquela série.
previous_load_kg	real	Fotografia da carga anterior no momento do registro — permite mostrar “+2,5 kg” sem recalcular histórico.
load_kg	real	A carga efetivamente usada. É o dado que o app existe para guardar.
completed_at	integer (ms)	Momento da conclusão; ordena a curva de progressão.
sync_status	text	<code>pending</code> · <code>synced</code> · <code>deleted</code> — é a fila de sincronização (adiante).

Três índices acompanham as consultas reais da interface: por `session_date` (treino do dia), por `exercise_id + completed_at` (curva de progressão) e por `sync_status` (varredura da fila).

progress_photos — fotos de progresso

COLUNA	TIPO	POR QUE PRECISA ESTAR NO APARELHO
id	text (PK)	UUID do cliente, reaproveitado como nome do arquivo no bucket.
user_id	text	Dono da foto; vira a pasta do arquivo no Storage.
local_uri	text	Caminho do arquivo no diretório de documentos do app — é o que a galeria exibe, sempre.
remote_path	text · nulo	Preenchido só depois do upload; nulo significa “ainda só existe aqui”.
taken_at	integer (ms)	Data de captura, usada para agrupar e ordenar a galeria.
note	text · nulo	Observação opcional do usuário.
sync_status	text	<code>pending</code> · <code>synced</code> .

A reatividade das telas vem do próprio banco: o SQLite é aberto com `enableChangeListener` e as telas consultam por `useLiveQuery`. Marcar uma série reexecuta as consultas afetadas e atualiza, no mesmo instante, o progresso do exercício, a sequência de dias, a faixa semanal e a curva — sem estado global duplicado e sem “puxar para atualizar”.

Fila de ações

Não existe uma tabela separada de “operações pendentes”. **A fila é uma coluna da própria linha** (`sync_status`): o registro que o usuário acabou de criar já é, ao mesmo tempo, o dado que a tela mostra e o item que falta enviar. Isso evita manter duas verdades sobre o mesmo fato e elimina a chance de a fila e o dado divergirem.

O caso delicado é a **exclusão**. Se o usuário desmarca uma série que já foi para a nuvem e o app apagasse a linha local, não sobraria nada para dizer ao servidor que aquele registro morreu — e a próxima leitura o traria de volta. Por isso a exclusão de um registro já sincronizado não apaga: marca a linha como `deleted` (um **tombstone**), que fica invisível para toda a interface e só é removida de vez depois que o servidor confirmar a exclusão.

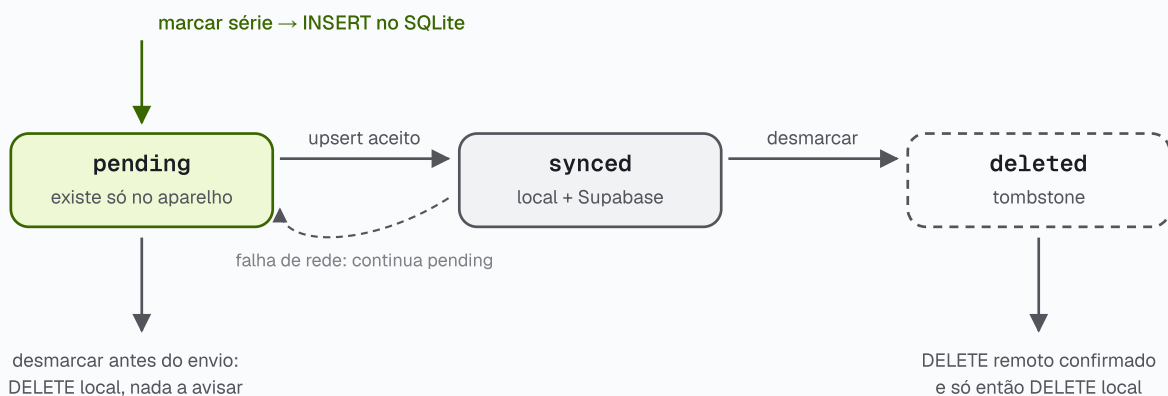


Figura 2. Ciclo de vida de um registro de série. Linhas cheias são transições confirmadas; a tracejada é o caminho de falha — nenhuma delas bloqueia a interface.

As fotos seguem a mesma lógica com um passo a mais: a captura sai do cache da câmera (que o sistema operacional pode limpar a qualquer momento) e é **movida para o diretório de documentos do app** antes de a linha ser gravada. Só depois disso o registro entra na fila como `pending`.

04 Backend e sincronização

O backend é o Supabase — Postgres, Auth e Storage no mesmo projeto. O aplicativo mobile fala com ele por duas vias distintas: **envia** o que produziu offline e **lê** o que foi produzido no aplicativo web.

Modelagem de dados remota

TABELA	COLUNAS PRINCIPAIS	ORIGEM E PAPEL
auth.users	id, email	Gerenciada pelo Supabase Auth. Todas as tabelas do app referenciam esse <code>id</code> com <code>on delete cascade</code> .
set_logs	id (uuid do cliente), user_id, session_date, exercise_id, exercise_name, set_number, target_reps, previous_load_kg, load_kg, completed_at	Espelho remoto da tabela local. Escrita exclusivamente pelo mobile.
progress_photos	id, user_id, storage_path, taken_at, note	Metadados da foto; o binário vive no bucket. Escrita pelo mobile.
body_assessments	id, user_id, pdf_path, status, extracted_data (jsonb), error_message	Avaliação física enviada em PDF pelo web. O <code>status</code> percorre <code>processing</code> → <code>pending_review</code> → <code>reviewed</code> (ou <code>error</code>). Lida pelo mobile.
nutrition_plans	id, assessment_id, user_id, calories_target, macros (jsonb), micros (jsonb), diet_guidance, model	Plano gerado por IA a partir da avaliação revisada, no backend do web. Lido pelo mobile.
storage: assessments	<user_id>/<arquivo>.pdf	Bucket privado dos laudos em PDF.
storage: progress-photos	<user_id>/<photo_id>.jpg	Bucket privado das fotos de progresso enviadas pelo mobile.

O esquema é versionado em arquivos de migration SQL no repositório (`supabase/migrations/`), o que mantém o banco reproduzível e revisável junto com o código.

Estratégia de sincronização

A sincronização é **por gatilhos, não por intervalo** — nada de polling consumindo bateria. O envio é tentado em quatro momentos:

- **Ao montar a área logada**, isto é, sempre que o app abre com sessão válida.

- Quando o NetInfo anuncia que a conexão voltou (assinatura de eventos de rede).
- Quando o app volta do segundo plano para o primeiro (evento do AppState).
- Quando o usuário aperta “sincronizar agora” em Configurações — a válvula manual para quem quer ver o número de pendências zerar.

Os três primeiros gatilhos ficam concentrados em um único hook (`useWorkoutSync`) montado no layout da área logada, com trava de reentrada para que dois gatilhos simultâneos não disparem duas rodadas ao mesmo tempo.

IMEDIATO — SEM REDE



DEPOIS — QUANDO HOUVER CONEXÃO

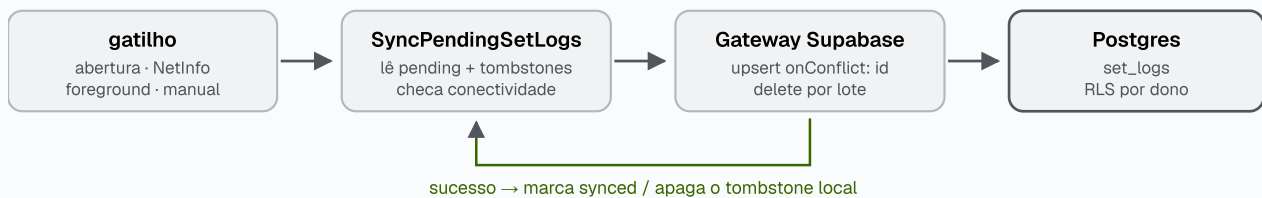


Figura 3. Caminho de escrita. A faixa de cima acontece em milissegundos e sem rede; a de baixo acontece quando dá, e falhar nela não custa nada ao usuário.

No sentido inverso — **nuvem** → **aparelho** — os dados de avaliação e do plano nutricional são buscados sob demanda, com refetch a cada vez que a tela ganha foco. Quem enviou o laudo no computador vê a avaliação aparecer no celular ao voltar para a tela, sem gesto de atualizar. Cada tela expressa três estados explícitos: carregando, erro e pronto.

Tratamento de conflitos

A política adotada é “o registro pertence a quem o criou, e o último envio vence” (**last-write-wins** por linha), sustentada por quatro decisões:

- **Identidade gerada no cliente.** O UUID nasce no aparelho, então o mesmo registro tem o mesmo `id` nos dois bancos. O envio é um `upsert onConflict: id`: reenviar dez vezes produz exatamente o mesmo resultado de enviar uma.
- **Escrita de mão única.** Séries e fotos só são escritas pelo aplicativo mobile; nenhum processo do servidor edita esses registros. Logo, o servidor nunca tem uma versão mais nova para disputar com o aparelho.
- **Exclusão com tombstone.** Uma série apagada offline não “ressuscita” na próxima sincronização, porque a exclusão viaja como intenção registrada e só some do aparelho depois de confirmada.

■ **Ordem e falha segura.** Primeiro os pendentes, depois as exclusões; e a rodada inteira falha em silêncio — o que não subiu continua `pending` para a próxima janela. Nenhum erro de rede interrompe o usuário no meio do treino.

O limite conhecido dessa escolha é o cenário de **dois aparelhos com a mesma conta editando a mesma série**: o último a sincronizar sobrescreve o outro. Como o produto é de uso individual e a sessão de treino acontece em um aparelho por vez, o custo de um relógio vetorial ou de uma coluna `updated_at` com resolução por versão não se justifica hoje — e a porta fica aberta, já que a resolução é feita em um único ponto do código (o gateway de sincronização).

Autenticação e permissões

Sim, o app tem login: **e-mail e senha via Supabase Auth**. A sessão é o que alimenta o guarda de rotas da seção 2 — não há “modo visitante”.

A persistência da sessão precisou de um cuidado específico. O armazenamento seguro do sistema (Keychain no iOS, Keystore no Android) tem limite prático de cerca de 2 KB por valor, insuficiente para a sessão do Supabase. A solução implementada é o padrão **LargeSecureStore**: gera-se uma chave AES-256, ela é guardada no armazenamento seguro do sistema e a sessão é gravada **cifrada** (AES-CTR) no armazenamento comum. A credencial nunca toca o disco em texto puro. O **refresh** automático do token só roda com o app em primeiro plano.

No banco, **todas as tabelas têm Row Level Security habilitada**, com políticas de dono para as quatro operações. O mesmo vale para os arquivos: as políticas de Storage comparam a **primeira pasta do caminho** com o identificador do usuário autenticado, de modo que ninguém lê o PDF nem a foto de outra pessoa mesmo conhecendo o caminho.

```
-- padrão aplicado a set_logs, progress_photos, body_assessments e nutrition_plans
alter table set_logs enable row level security;

create policy "set_logs_select_own" on set_logs
  for select using (auth.uid() = user_id);
create policy "set_logs_insert_own" on set_logs
  for insert with check (auth.uid() = user_id);

-- arquivos: a pasta raiz do objeto precisa ser o id do usuário
create policy "progress_photos_storage_select_own" on storage.objects
  for select using (
    bucket_id = 'progress-photos'
    and (storage.foldername(name))[1] = auth.uid()::text
  );
```

Os buckets são privados; o aplicativo usa a chave anônima pública do projeto, e é a RLS — não o cliente — que define o que cada sessão enxerga. Não existe chave de serviço embarcada no app.

05 Hardware e sensores

Uso da câmera

A câmera aparece em **dois fluxos distintos**, cada um com sua tela dedicada de tela cheia.

Fluxo 1 — Foto de progresso

A foto de progresso é a evidência visual que acompanha os números da avaliação: o mesmo enquadramento, a cada poucas semanas. O usuário chega pela galeria (`/fotos`) ou pelo Perfil, e a captura é gravada **antes de qualquer chamada de rede**:

- 1 A tela pede a permissão de câmera com uma explicação em português — e um caminho de saída (“Agora não”) para quem não quiser conceder.
- 2 A captura sai da câmera com qualidade 0,85 e cai no diretório de cache do sistema.
- 3 O arquivo é **movido do cache para o diretório de documentos do app**, em `progress-photos/<uuid>.jpg`. O cache pode ser esvaziado pelo sistema operacional a qualquer momento; o diretório de documentos, não.
- 4 Uma linha é gravada no SQLite com `local_uri`, data e `sync_status = pending`. A galeria já mostra a foto neste ponto, com ou sem internet.
- 5 Havendo conexão, o mesmo caso de uso já tenta o upload; não havendo, a foto entra na fila e sobe no próximo gatilho de sincronização.
- 6 O upload envia os bytes para `progress-photos/<user_id>/<photo_id>.jpg` e faz o `upsert` da linha remota. Só então `remote_path` é preenchido e o status vira `synced`.

Repare que a galeria **sempre lê o arquivo local**, mesmo depois de sincronizada: a nuvem é backup, não a origem da exibição. Por isso rolar as fotos é instantâneo e não consome dados. O indicador de sincronização em cada foto existe justamente para tornar visível a diferença entre “está no meu celular” e “está salva na nuvem”.

Fluxo 2 — Leitura de QR code do aparelho

Achar o exercício certo numa lista, com o celular na mão e o aparelho ocupado, é fricção pura. O leitor resolve isso: cada aparelho da academia recebe um QR code contendo `pxgym://treino/<exercicioId>` (ou o identificador puro), e apontar a câmera abre a tela daquele exercício diretamente, pronta para registrar a carga. O leitor trava após a primeira leitura válida para não navegar duas vezes, e um código desconhecido produz uma mensagem clara em vez de uma tela em branco. Esse fluxo é **totalmente offline** — a leitura e a navegação não dependem de rede.

Permissões. Os textos de permissão são declarados no `app.json` em português e explicam a finalidade concreta: *“O Px GYM usa a câmera pra registrar fotos de progresso e ler QR codes dos aparelhos.”* No Android, `CAMERA`, `ACCESS_COARSE_LOCATION` e `ACCESS_FINE_LOCATION` são declaradas explicitamente. Nenhuma permissão é pedida na abertura do app: cada uma é solicitada na tela em que faz sentido, com a explicação à vista.

Uso da geolocalização

A localização serve a um caso de uso único e concreto: **encontrar academias perto de onde o usuário está agora**. É o cenário de quem viaja, mudou de bairro ou quer treinar perto do trabalho — sem localização, a única alternativa seria digitar um endereço, que é justamente o tipo de fricção que o app evita.

A captura é **pontual e apenas em primeiro plano**. Não há rastreamento em segundo plano: nenhuma permissão do tipo “sempre”, nenhum serviço de localização contínua e, portanto, nenhum consumo de bateria fora do momento em que a tela de academias está aberta. A posição é lida quando a tela abre (ou quando o usuário toca em “tentar de novo”) e descartada em seguida — ela não é gravada no SQLite nem enviada ao Supabase.

- **Precisão “balanceada”,** e não a máxima: para um raio de busca de 4 km, a precisão de alguns metros do GPS fino seria desperdício de tempo e bateria.
- **Plano B para o fix que não sai.** GPS frio, ambiente fechado ou emulador podem fazer a leitura de posição atual falhar. Nesse caso o app cai na última posição conhecida do sistema, sem limite de idade — para um raio de 4 km ela serve tão bem quanto uma leitura nova, e é melhor que uma tela de erro.
- **Fonte de dados sem chave e sem custo:** as academias vêm do OpenStreetMap pela Overpass API, consultando `leisure=fitness_centre` e `amenity=gym` ao redor do ponto. Um adaptador alternativo para a Google Places API está implementado e desativado, pronto para ser ligado se um dia houver faturamento — a tela não muda, só o adaptador.
- **Mapa nativo por plataforma:** Apple Maps no iOS e MapLibre com tiles do OpenStreetMap no Android, resolvidos por extensão de arquivo de plataforma, sem `if` espalhado pela tela.

Como toda a tela depende de rede (Overpass) e de sensor (GPS), ela trata os dois fracassos separadamente: “não consegui sua localização” e “não consegui buscar academias” são mensagens diferentes, com ação de repetir.

06 Arquitetura e padrões de código

O projeto é um monorepo com dois aplicativos e três pacotes compartilhados. A regra que organiza tudo: **o domínio não conhece nem o React, nem o Expo, nem o Supabase.**

Organização do projeto

px-gym/	
├─ apps/	
│ └─ mobile/	Expo · React Native
│ │ └─ src/app/	rotas (Expo Router) – só composição de tela
│ │ └─ src/components/	UI reutilizável (cartões, gráficos, barras)
│ │ └─ src/db/	SQLite: schema, cliente e repositórios Drizzle
│ │ └─ src/lib/	integrações e composition roots
│ │ └─ drizzle/	migrations do banco local
│ └─ web/	Next.js – envio e revisão do laudo, plano por IA
├─ packages/	
│ └─ core/	domínio puro – sem React, Expo ou Supabase
│ │ └─ entities/	SetLog, ProgressPhoto, BodyAssessment...
│ │ └─ value-objects/	Load, Weight, BodyFat, Macro, GeoPoint
│ │ └─ use-cases/	RegisterSet, SyncPendingSetLogs, SaveProgressPhoto...
│ │ └─ services/	progressão de carga, sequência de dias, medidas
│ │ └─ ports/	interfaces que a infraestrutura implementa
│ │ └─ testing/	fakes em memória usados pelos testes
│ └─ db/	adaptadores Supabase (repositórios e gateway)
│ └─ tokens/	tokens de design compartilhados entre web e mobile
└─ supabase/migrations/	esquema remoto versionado em SQL

Figura 4. Estrutura de diretórios. As duas aplicações consomem os mesmos pacotes de domínio e de design.

Separação de responsabilidades

A separação segue **ports & adapters** (arquitetura limpa). O domínio declara as interfaces de que precisa; cada tecnologia entra como implementação intercambiável:

CAMADA	O QUE CONTÉM	DO QUE ELA NÃO PODE DEPENDER
Domínio packages/core	Entidades, objetos de valor, casos de uso, serviços de cálculo e os ports : SetLogRepository , WorkoutSyncGateway , ProgressPhotoUploader , NearbyGymsFinder , ConnectivityStatus , Clock , IdGenerator .	De nada. Não importa React, React Native, Expo, Drizzle nem Supabase — por isso roda em Node puro nos testes.
Banco local mobile/src/db	Esquema Drizzle, migrations e os repositórios que implementam os ports contra o SQLite, além da conversão linha ↔ entidade.	De componentes ou telas. Nenhuma consulta SQL vive dentro de uma tela.
Backend packages/db	Cliente Supabase, repositórios de avaliação e plano, gateway de sincronização e mapeadores.	De React e do domínio de UI; é compartilhado com o app web.
Composição mobile/src/lib	Os composition roots que instanciam cada caso de uso com seus adaptadores concretos, mais os hooks que ligam isso às telas.	É o único lugar onde “SQLite” e “Supabase” se encontram na mesma linha.
Apresentação mobile/src/app · components	Telas e componentes visuais: leem por hooks , escrevem chamando casos de uso.	Não faz consulta SQL, não chama a API do Supabase, não formula regra de progressão.



as setas são a direção das dependências: nada dentro do domínio aponta para fora
trocar SQLite, Supabase ou a fonte de academias não toca em uma linha de regra

Figura 5. Direção das dependências. Nos testes, os adaptadores reais são substituídos por implementações falsas em memória — é o que permite testar sincronização e progressão de carga sem banco, sem rede e sem emulador.

O ganho prático dessa separação é testabilidade: a suíte do domínio tem **77 testes** cobrindo progressão de carga, sequência de dias, atividade da semana, objetos de valor, registro de série e — o mais importante — o comportamento da sincronização (offline, nada a enviar, envio bem-sucedido, tombstones), tudo rodando em Node puro, em segundos.

Gerenciamento de estado de conexão

O estado de rede é tratado em **dois níveis diferentes**, de propósito.

Como decisão de domínio. A conectividade entra no núcleo como um **port** (`ConnectivityStatus`), e o caso de uso de sincronização consulta esse port antes de tentar qualquer envio — se estiver offline, ele devolve o resultado `offline` sem gastar requisição. A implementação real usa o `NetInfo`, com um detalhe que importa: enquanto o sistema ainda está sondando a rede, o indicador de “internet alcançável” vem indefinido, e o app só bloqueia o envio quando ele afirma **explicitamente** que não há internet. Nos testes, esse mesmo port vira uma implementação falsa que devolve o que o cenário pedir.

Como reação a eventos. Um único **hook** montado no layout da área logada assina o `NetInfo` e o `AppState`: conexão restabelecida ou app voltando ao primeiro plano disparam uma rodada de sincronização em segundo plano, com trava de reentrada.

Do lado da interface, a escolha é **não interromper**. Falha de sincronização não vira alerta nem modal — quem está no meio de uma série não precisa lidar com um erro de rede que o app já sabe contornar. Em vez disso, o estado fica disponível onde o usuário pode agir sobre ele:

- **Configurações** mostra, ao vivo, quantos registros e fotos estão pendentes (consulta reativa ao SQLite) e oferece o botão de sincronizar agora.
- **A galeria de fotos** marca cada item com um indicador de já sincronizado ou ainda local.
- **Telas que dependem de leitura remota** (Avaliações, Nutrição, Academias) têm estado de erro próprio, com ícone de “sem nuvem”, explicação e ação de repetir.

Estado atual do projeto. Estão implementados: autenticação com sessão cifrada e rotas protegidas, registro de série offline com progressão de carga, sincronização por gatilhos com tombstones, fotos de progresso com upload diferido, leitor de QR, busca de academias por GPS, painéis de progresso corporal e macros, e a suíte de 77 testes de domínio. A rotina de exercícios do dia ainda vem de um conjunto fixo em código — a montagem de treinos pelo usuário é o próximo passo natural, e por já existir o repositório de treino como *port*, ela entra sem alterar a estratégia offline descrita aqui.